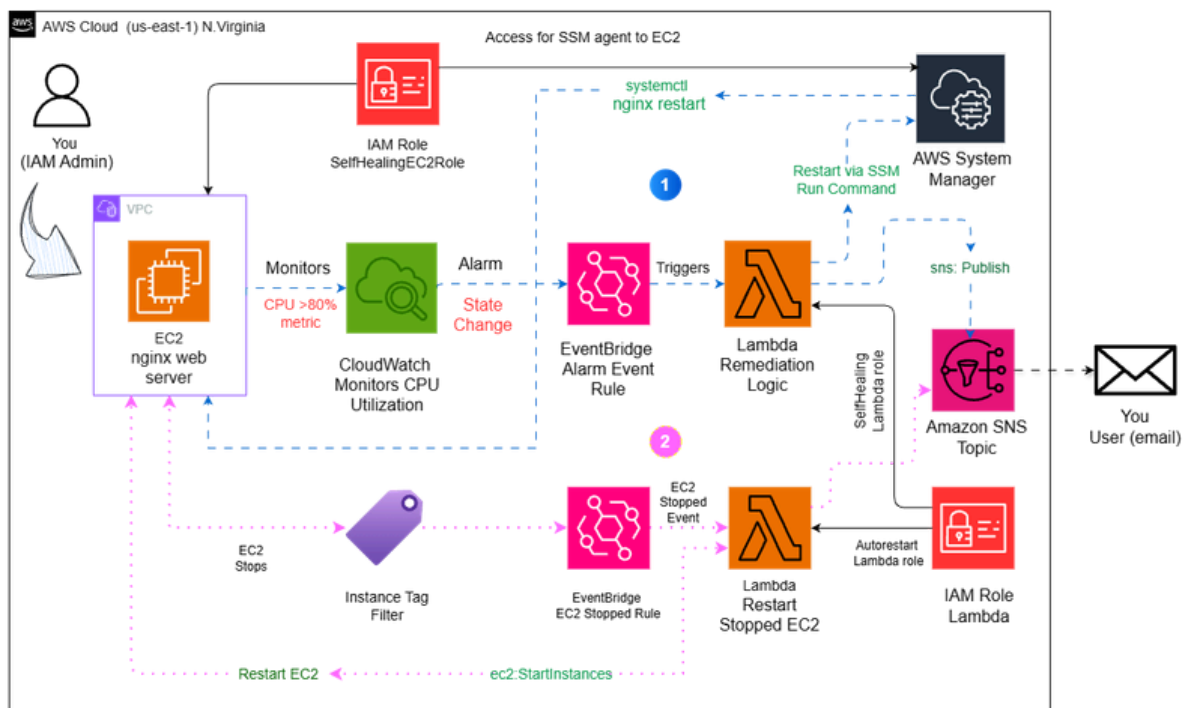


Kareshma
Rajaananthapadmanaban

Event Driven Self Healing DevOps Monitoring System on AWS

Automate EC2 failure detection and recovery using
EventBridge and Lambda.

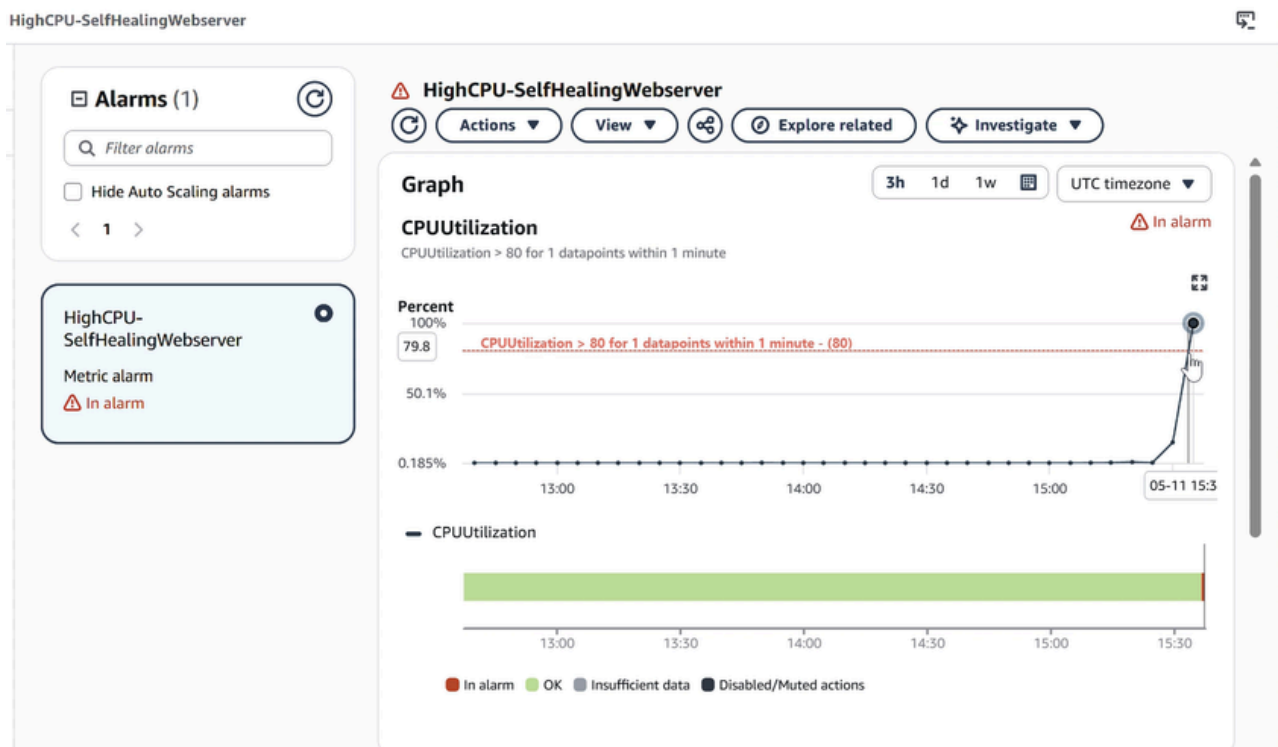


Building a Self-Healing Infrastructure on AWS



Kareshma Rajaananthapadmanaban

Project Type: Event-Driven DevOps Automation Focus Areas: AWS, DevOps Automation, Observability, Self-Healing System





Introduction

This project demonstrates how to build a self-healing infrastructure using event driven AWS services. The system detects failures in a web server automatically and remediates the issue without manual intervention. The pipeline combines monitoring, event routing, serverless remediation, and automated notifications.

Problem Statement

In production environments, service failures such as crashed web servers or accidentally stopped instances can cause downtime. Manual troubleshooting increases response time and operational overhead. This project addresses that problem by implementing an automated remediation pipeline that detects anomalies and fixes them automatically

Solution Architecture

The architecture uses monitoring, event routing, and serverless automation to detect and remediate failures. When a service failure or resource state change occurs, monitoring tools trigger events that invoke Lambda functions responsible for corrective actions.



Architecture Overview

Architecture Workflow:

- 1.The nginx web server runs on an EC2 instance.
- 2.CloudWatch monitors CPU utilization and service health.
- 3.When CPU usage exceeds the defined threshold or service failure occurs, a CloudWatch alarm is triggered.
- 4.EventBridge captures the alarm state change event.
- 5.EventBridge invokes the remediation Lambda function.
- 6.Lambda restarts nginx through Systems Manager Run Command.
- 7.SNS sends an email notification describing the remediation action to the user.
- 8.EventBridge detects EC2 stopped event and Lambda checks for AutoRestart=true tag.
- 9.Instance is restarted automatically and SNS notification is sent.

AWS Services Used

Service	Purpose
EC2	Hosts the nginx web server used for testing failures
CloudWatch	Monitors CPU metrics and triggers alarms
EventBridge	Routes events from alarms and EC2 state changes
Lambda	Executes automated remediation logic
Systems Manager (SSM)	Runs commands on EC2 instances
SNS	Sends email notifications
IAM	Controls permissions for EC2 and Lambda roles



Infrastructure Components

- EC2 Instance: SelfHealing-Webserver (t3.micro)

IAM Roles: -

- SelfHealingEC2Role
- SelfHealingLambdaRole
- AutoRestartLambdaRole

Monitoring: -

- CloudWatch Alarm: HighCPU-SelfHealingWebserver

Automation: -

- EventBridge Rule: HighCPU-Remediation-Rule
- EventBridge Rule: EC2-AutoRestart-Rule
- Lambda Function: SelfHealingRemediation
- Lambda Function: AutoRestartInstance

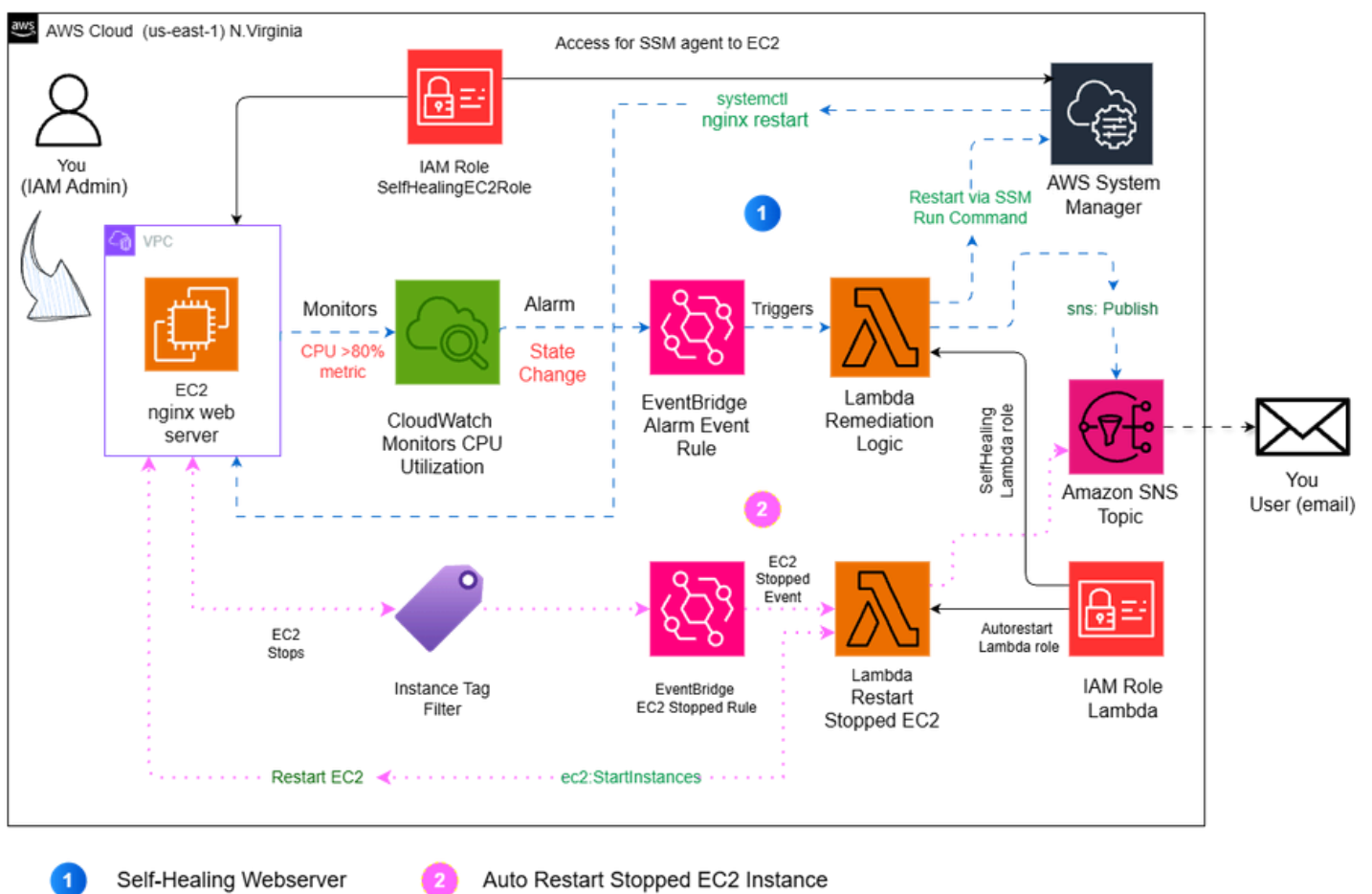
Notifications: -

- SNS Topic: SelfHealingAlerts



Architecture Diagram

Event Driven Self Healing monitoring system on AWS





Project overview and goals

In this project, I'm building an event-driven self-healing system on AWS that detects nginx web server failures running on an EC2 instance and automatically restarts the service using AWS Lambda and Systems Manager. The system uses CloudWatch alarms and EventBridge rules to monitor failures and trigger remediation, while Amazon SNS sends notifications confirming that the issue was detected and automatically resolved.

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Amazon EC2, Amazon CloudWatch, Amazon EventBridge, AWS Lambda, Amazon SNS, AWS Systems Manager, and AWS Identity and Access Management. Key concepts I learned include building an event-driven automation workflow, monitoring infrastructure using CloudWatch alarms, triggering remediation with Lambda, and designing a self-healing system that automatically detects failures, restarts services, and sends alerts without manual intervention.

Time and challenges

This project took me approximately 1 hour and 30 minutes to complete. The most challenging part was troubleshooting an invalid instance ID error, which happened because my Lambda function was created in a different AWS region while the EC2 instance and monitoring resources were running in another region. Once I aligned all resources in the same region, the pipeline worked correctly.

Deploying the nginx Web Server on EC2

Setting up EC2 with IAM and nginx

In this step, I'm setting up an EC2 instance running an nginx web server and configuring an IAM role that allows AWS Systems Manager to securely access and manage the instance. I attach the role during launch and install nginx using a user data startup script so the web server starts automatically. This creates the base environment for the project, allowing me to monitor the server and later automate failure detection and recovery through the self-healing pipeline.

The screenshot displays the AWS Management Console interface for the EC2 service. At the top, the navigation bar shows the AWS logo, a search bar, and the current region (United States (N. Virginia)). The main content area is titled 'Instances (1/1)' and shows a single instance named 'SelfHealing-Webserver' with ID 'i-0c11607843c0f048d'. The instance is in the 'Running' state, using a 't3.micro' instance type, and has passed all status checks. Below the instance list, the details for 'i-0c11607843c0f048d (SelfHealing-Webserver)' are shown. The 'Instance summary' section includes the instance ID, IPv6 address (none), hostname type (ip-172-31-7-123.ec2.internal), and answer private resource DNS name (none). The 'Public IPv4 address' is 18.207.100.199, and the 'Private IPv4 addresses' is 172.31.7.123. The 'Instance state' is 'Running', and the 'Private IP DNS name (IPv4 only)' is ip-172-31-7-123.ec2.internal. The 'Instance type' is 't3.micro'.

Launch EC2 instance with startup script under user data

Deploying the nginx Web Server on EC2

Setting up EC2 with IAM and nginx

In this step, I'm setting up an EC2 instance running an nginx web server and co

```
$ Startup-script.sh
1  #This script runs automatically when your EC2 instance boots for the first time.
2
3  ##!/bin/bash tells the instance to run this as a shell script.
4  #!/bin/bash
5
6  #sudo dnf install -y nginx installs the nginx web server using the Amazon Linux 2023 package manager.
7
8  sudo dnf install -y nginx
9
10 #sudo systemctl start nginx starts the web server immediately.
11
12 sudo systemctl start nginx
13
14 #sudo systemctl enable nginx ensures nginx restarts automatically if the instance reboots.
15
16 sudo systemctl enable nginx
```

EC2 > Instances > Launch an instance

Select

Firewall (security groups) | Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

Security group name - *required*
launch-wizard

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and .-:/()@[]+=&:|!\$*

Description - *required* | Info
launch-wizard created 2026-05-09T14:39:49.224Z

Inbound Security Group Rules
▼ Security group rule 1 (TCP, 80, 0.0.0.0/0) Remove

Type Info	Protocol Info	Port range Info
HTTP	TCP	80

Source type Info	Source Info	Description - <i>optional</i> Info
Anywhere	Add CIDR, prefix list or security group 0.0.0.0/0	e.g. SSH for admin desktop

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only. ✕

Network set up - Security group



Setting up EC2 with IAM and nginx

Role name
Enter a meaningful name to identify this role.

Maximum 64 characters. Use alphanumeric and '+,=,_,@,-.' characters.

Description
Add a short explanation for this role.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/\[\]!#\$%^&*(){}~`

Step 1: Select trusted entities

Trust policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "",
6       "Effect": "Allow",
7       "Principal": {
8         "Service": "ec2.amazonaws.com"
9       },
10      "Action": "sts:AssumeRole"
11    }
12  ]
13 }
```

Step 2: Add permissions

Permissions policy summary

Policy name	Type	Attached as
AmazonSSMManagedInstanceCore	AWS managed	Permissions policy

IAM Role setup

Dashboard > IAM > Roles > SelfHealingEC2Role

Role SelfHealingEC2Role created. [View role](#)

SelfHealingEC2Role [Info](#) [Delete](#)

Allows EC2 instances to call AWS services like CloudWatch and Systems Manager on your behalf.

Summary [Edit](#)

Creation date May 06, 2026, 19:43 (UTC+05:30)	ARN arn:aws:iam::799990344483:role/SelfHealingEC2Role	Instance profile ARN arn:aws:iam::799990344483:instance-profile/SelfHealingEC2Role
Last activity -	Maximum session duration 1 hour	

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (1/1) [Info](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Filter by Type: All types

Policy name	Type	Attached entities
AmazonSSMManagedInstanceCore	AWS managed	3

IAM Role

IAM role permissions for Systems Manager

The IAM role allows the instance to securely communicate with AWS Systems Manager so it can receive and execute remote management commands. In this project, it enables the pre-installed SSM Agent on the EC2 instance to register with Systems Manager and accept commands without requiring SSH access. This permission allows services like Lambda to remotely run commands on the instance, such as restarting the nginx web server during automated failure recovery.

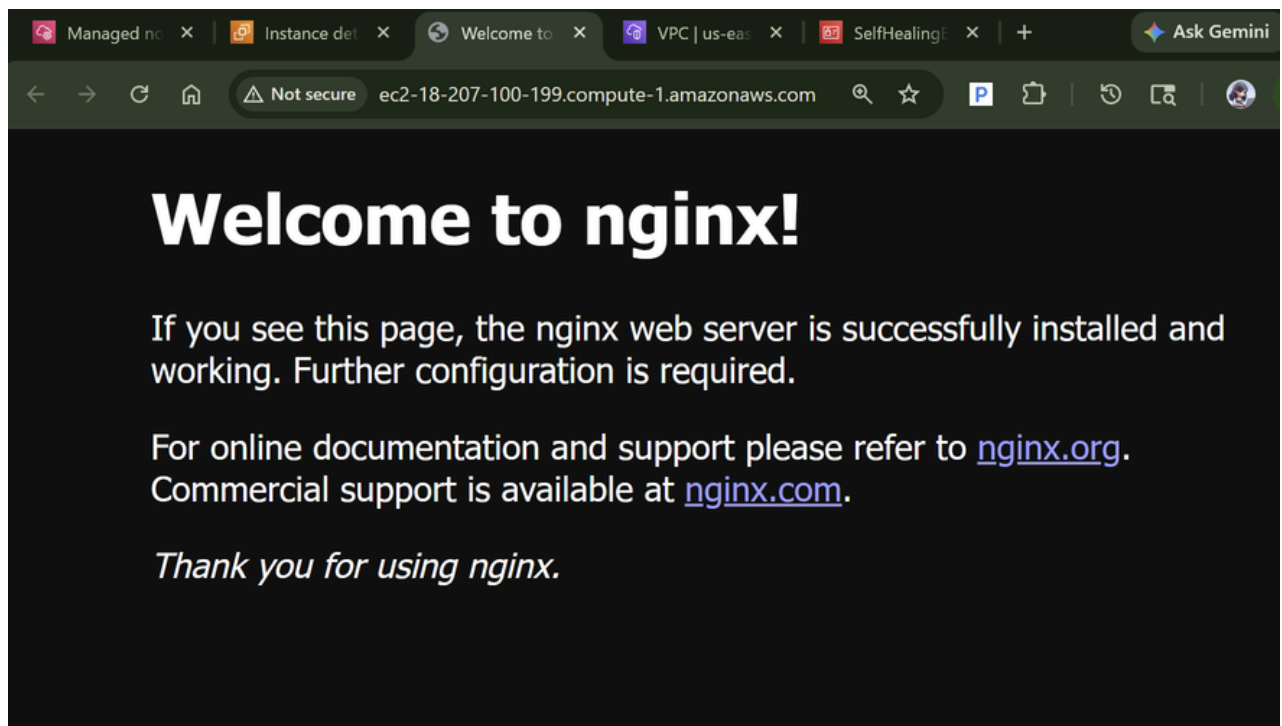
The screenshot shows the AWS Systems Manager console. On the left is a navigation menu with options like 'Review node insights', 'Explore nodes', 'Diagnose and remediate', 'Just-in-time node access', 'Settings', 'Node Tools' (including Compliance, Distributor, Fleet Manager, Hybrid Activations, Inventory, Patch Manager, Run Command, Session Manager, State Manager), and 'Change Management Tools' (Automation, Change Calendar). The main area is titled 'Fleet Manager' and shows a notification about unmanaged Amazon EC2 instances. Below this, the 'Managed Nodes (1)' section displays a table with one node.

Node ID	Node Status	Name	Platform	Operating System	Resource
i-0c11607...	Running	SelfHealing-Webserver	Linux	Amazon L...	EC2

SSM Fleet Manager

This is a close-up of the 'Managed Nodes (1)' table. It shows a single node with the following details:

for...	Operat...	Resource...	Source ID	Ping status	Agent versi...	Image ID	EC2 insta...
ix	Amazon L...	EC2 instance	-	Online	3.3.4108.0	ami-0a59ec9...	Open EC2 ...



nginx web page

Configuring Alerts with SNS and CloudWatch

Setting up the notification and monitoring layer

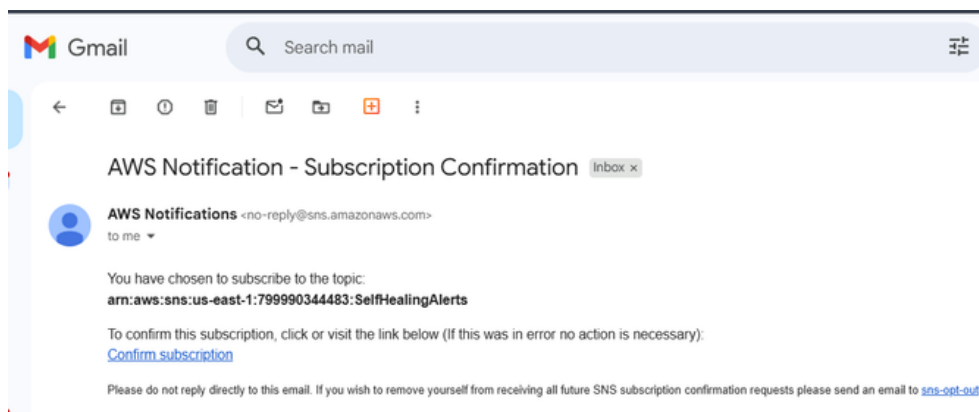
In this step, I'm setting up an alerting and monitoring layer using Amazon SNS and CloudWatch so that I can automatically detect abnormal activity on my EC2 instance and receive notifications when a threshold is triggered. I create an SNS topic with an email subscription for alerts and configure a CloudWatch alarm to monitor CPU utilization. When CPU usage exceeds the defined limit, the alarm sends a notification through SNS, enabling the monitoring pipeline to react to potential failures.



The screenshot shows the Amazon SNS console for the topic 'SelfHealingAlerts'. The left sidebar shows the navigation menu with 'Topics' selected. The main content area shows the topic details and a list of subscriptions. The subscription list shows one subscription with ID '63691957-77b1-4930-9a...', endpoint 'kareshma2909@gm...', status 'Confirmed', and protocol 'EMAIL'.

ID	Endpoint	Status	Protocol
63691957-77b1-4930-9a...	kareshma2909@gm...	Confirmed	EMAIL

SNS Topic with subscription



Simple Notification Service

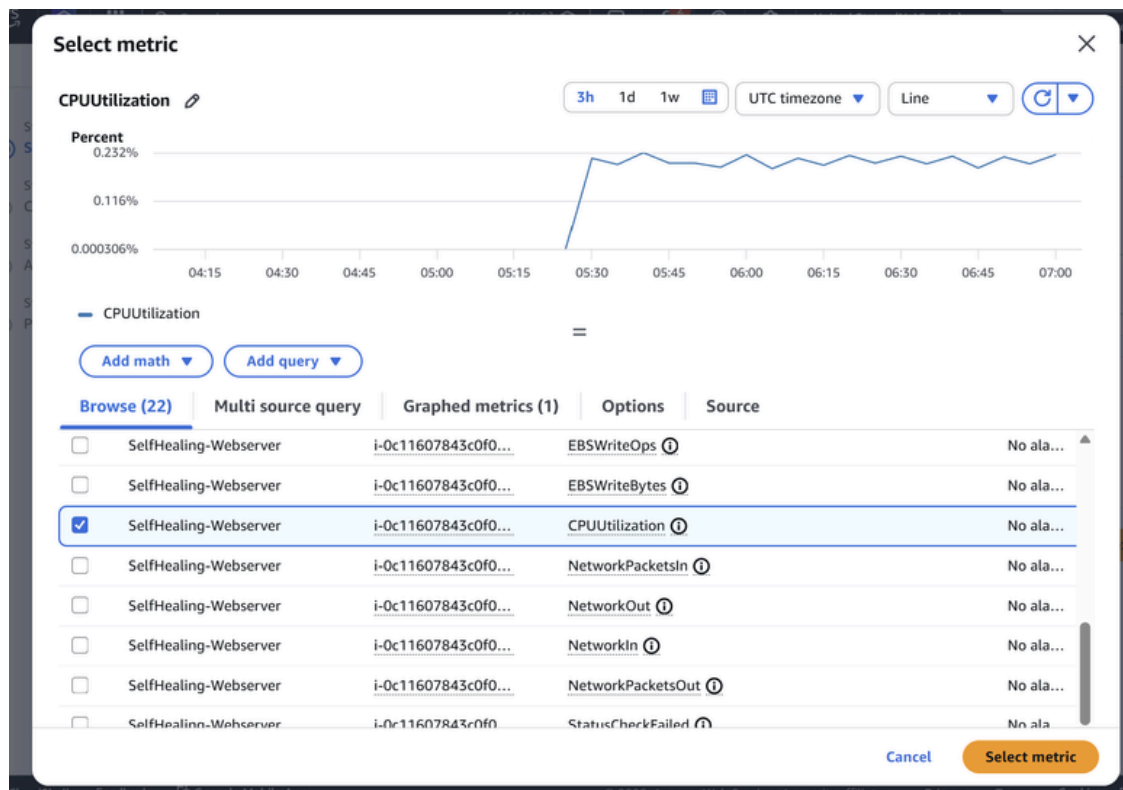
Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts:63691957-77b1-4930-9a20-b8b0e4dd3e18

If it was not your intention to subscribe, [click here to unsubscribe](#).



Metric of EC2 CPU Utilization

Alarm trigger conditions and actions

The alarm triggers when the EC2 instance's CPU utilization exceeds 80% during the 1-minute evaluation period. In this project, I configured the alarm to evaluate one datapoint, so a single reading above the threshold immediately changes the alarm state to In Alarm. When this condition is met, CloudWatch publishes a notification to the SelfHealingAlerts SNS topic, which then sends an email alert to my subscribed address to notify me that the monitoring system detected abnormal activity.

CloudWatch > Alarms > Create alarm

Step 1
Specify metric and conditions

Step 2
Configure actions

Step 3
Add alarm details

Step 4
Preview and create

Preview and create

Step 1: Specify metric and conditions

Metric

Graph
This alarm will trigger when the blue line goes above the red line for 1 datapoints within 1 minute.

Percent
80%
40%
0.000306%

05:00 06:00 07:00

— CPUUtilization

Namespace
AWS/EC2

Metric name
CPUUtilization

Instanceid
i-0c11607843c0f048d

Instance name
SelfHealing-Webserver

Statistic
Average

Period
1 minute

Edit

Configure alarm for CloudWatch

CloudWatch > Alarms > Create alarm

Conditions

Threshold type
Static

Whenever CPUUtilization is
Greater (>)

than...
80

▼ Additional configuration

Datapoints to alarm
1 out of 1

Missing data treatment
Treat missing data as missing

Step 2: Configure actions

Actions

Notification
When In alarm, send a notification to "SelfHealingAlerts"

Step 3: Add alarm details

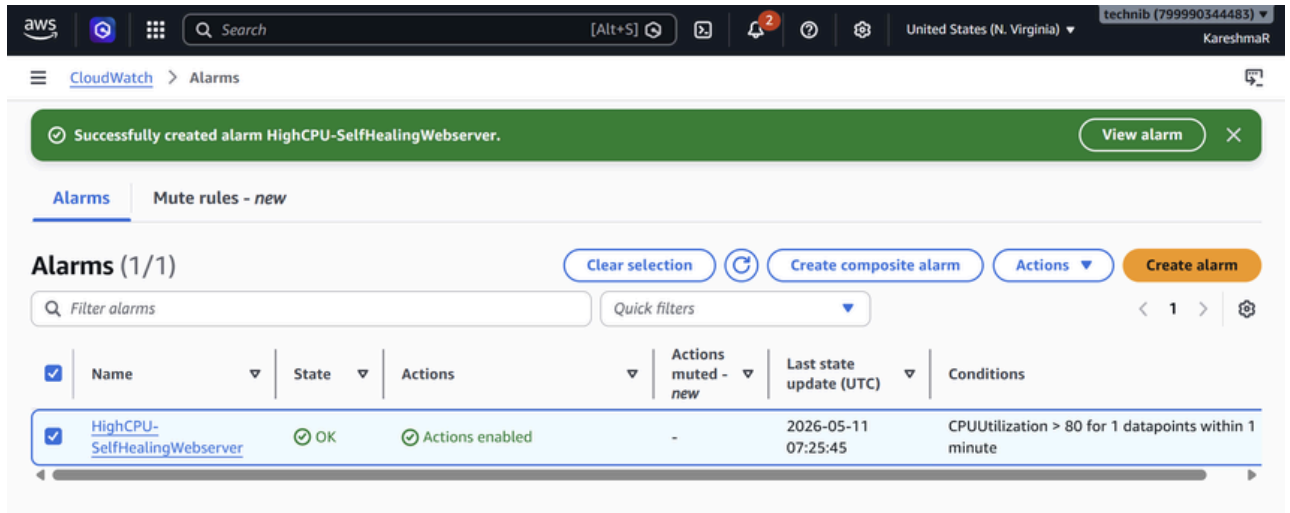
Alarm details

Name
HighCPU-SelfHealingWebserver

Description
-

► Tags (0)

Cancel Previous Create alarm



CloudWatch alarm list

Writing the Lambda Remediation Function

Building automated remediation logic in Python

In this step, I'm building the automated remediation component using AWS Lambda and IAM so that the system can respond to failures without manual intervention. I create a Lambda function with the required permissions to interact with Systems Manager and SNS. The function receives alarm events, identifies the affected EC2 instance, and remotely executes a command to restart the nginx service. After performing the remediation, it also sends a notification confirming that the system automatically detected and resolved the issue.

Lambda Remediation Logic

The Lambda function performs the following tasks:

- Receives CloudWatch alarm events
- Extracts instance ID from event payload
- Sends SSM Run Command to restart nginx
- Publishes notification to SNS

Example command executed via SSM: `sudo systemctl restart nginx`



☰ IAM > Roles > SelfHealingLambdaRole ⓘ

✔ Role SelfHealingLambdaRole created. View role ✕

SelfHealingLambdaRole Info

Allows Lambda functions to call AWS services on your behalf. Delete

Summary

Creation date
May 11, 2026, 17:00 (UTC+05:30)

Last activity
-

ARN
 `arn:aws:iam::799990344483:role/SelfHealingLambdaRole`

Maximum session duration
1 hour

Edit

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (1/1) Info

You can attach up to 10 managed policies.

Filter by Type All types < 1 > ⚙️

☒	Policy name	Type	Attached entities
☒	AWSLambdaBasicExecutionRole	AWS managed	1

Lambda execution role and inline policy

☰ IAM > Roles > SelfHealingLambdaRole > Create policy

Step 1

☒ **Specify permissions**

Step 2

☐ Review and create

Specify permissions Info

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

Visual **JSON** Actions

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "ssm:SendCommand",  
7       "Resource": "*"   
8     },  
9     {  
10      "Effect": "Allow",  
11      "Action": "sns:Publish",  
12      "Resource": "arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts"  
13    }  
14  ]  
15 }
```

Edit statement

Add actions

Choose a service

Included

[Systems Manager](#)

Available

- [AI Operations](#)
- [AMP](#)
- [API Gateway](#)
- [API Gateway V2](#)
- [ARC Region switch](#)
- [ARC Zonal Shift](#)
- [ASC](#)
- [Access Analyzer](#)



The screenshot shows the 'Create policy' page in the AWS IAM console. The breadcrumb trail is IAM > Roles > SelfHealingLambdaRole > Create policy. The page has two steps: 'Specify permissions' and 'Review and create'. The 'Review and create' step is active. The 'Policy details' section shows the 'Policy name' as 'SelfHealingRemediationPolicy'. The 'Permissions defined in this policy' section shows a table with two permissions: 'SNS' and 'Systems Manager'. The 'SNS' permission is 'Limited: Write' on the resource 'region] string like [us-east-1, TopicName] string like [SelfHealingAlerts'. The 'Systems Manager' permission is 'Limited: Write' on 'All resources'. The 'Request condition' for both is 'None'. The 'Show remaining 467 services' toggle is turned on. The 'Create policy' button is highlighted in orange.

Step 1
Specify permissions

Step 2
Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name
Enter a meaningful name to identify this policy.

SelfHealingRemediationPolicy

Maximum 128 characters. Use alphanumeric and '+*,@-_' characters.

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

Q Search

Allow (2 of 469 services) Show remaining 467 services

Service	Access level	Resource	Request condition
SNS	Limited: Write	region] string like [us-east-1, TopicName] string like [SelfHealingAlerts	None
Systems Manager	Limited: Write	All resources	None

Cancel Previous **Create policy**

Lambda role inline policy

The screenshot shows the 'SelfHealingLambdaRole' page in the AWS IAM console. The breadcrumb trail is IAM > Roles > SelfHealingLambdaRole. A green banner at the top says 'Policy SelfHealingRemediationPolicy created.' The 'Summary' section shows the 'Creation date' as 'May 11, 2026, 17:00 (UTC+05:30)' and the 'Last activity' as '-'. The 'ARN' is 'arn:aws:iam::799990344483:role/SelfHealingLambdaRole' and the 'Maximum session duration' is '1 hour'. The 'Permissions' tab is selected, showing 'Permissions policies (2/2)'. The table lists two policies: 'AWSLambdaBasicExecutionRole' (AWS managed, 1 attached entity) and 'SelfHealingRemediationPolicy' (Customer inline, 0 attached entities).

Policy SelfHealingRemediationPolicy created.

SelfHealingLambdaRole

Allows Lambda functions to call AWS services on your behalf.

Summary

Creation date
May 11, 2026, 17:00 (UTC+05:30)

Last activity
-

ARN
arn:aws:iam::799990344483:role/SelfHealingLambdaRole

Maximum session duration
1 hour

Permissions Trust relationships Tags Last Accessed Revoke sessions

Permissions policies (2/2)

You can attach up to 10 managed policies.

Q Search Filter by Type All types

Policy name	Type	Attached entities
AWSLambdaBasicExecutionRole	AWS managed	1
SelfHealingRemediationPolicy	Customer inline	0

Lambda role with policy



IAM > Roles > SelfHealingLambdaRole

☒ [AWSLambdaBasicExecutionRole](#) AWS managed 1

AWSLambdaBasicExecutionRole [Copy JSON](#)

Provides write permissions to CloudWatch Logs.

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "logs:CreateLogGroup",  
8         "logs:CreateLogStream",  
9         "logs:PutLogEvents"  
10      ],  
11      "Resource": "*"  
12    }  
13  ]  
14 }
```

☒ [SelfHealingRemediationPolicy](#) Customer inline 0

SelfHealingRemediationPolicy [Copy JSON](#) [Edit](#)

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "ssm:SendCommand",  
7       "Resource": "*"  
8     },  
9     {  
10      "Effect": "Allow",  
11      "Action": "sns:Publish",  
12      "Resource": "arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts"  
13    }  
14  ]  
15 }
```

Lambda > Functions > Create function

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom** execution role under **More settings**.

Custom settings [Info](#)

Durable execution - *new* ☐
Build resilient, stateful applications with automatic failure recovery.

EC2 capacity provider - *new* ☐
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

Lambda function

Custom execution role

[Edit](#) ☒

Use custom execution role instead of new role with basic Lambda permissions (default).

IAM permissions

[SelfHealingLambdaRole](#)

The screenshot shows the AWS Lambda console for the function 'SelfHealingRemediation'. A green notification bar at the top states 'Successfully updated the function "SelfHealingRemediation"'. The function overview section shows the function name, a diagram icon, and a 'Layers' section with '(0)' layers. The 'Configuration' tab is selected, showing the 'General configuration' section. The 'Description' is empty, 'Memory' is 128 MB, 'Ephemeral storage' is 512 MB, 'Timeout' is 0 min 30 sec, and 'SnapStart' is None. The 'Function ARN' is 'arn:aws:lambda:ap-south-1:799990344483:function:SelfHealingRemediation'. The 'Last modified' time is '6 seconds ago'. The 'Configuration' section on the left lists 'General configuration', 'Triggers', 'Permissions', and 'Destinations'.

Lambda function configuration timeout

The screenshot shows the AWS Lambda console for the function 'SelfHealingRemediation' with the code editor open. The code is in Python and defines a lambda handler that processes an EventBridge event. The handler extracts the alarm name and instance ID from the event, sends an SSM Run Command to restart nginx on the affected instance, builds a remediation summary with a timestamp, and publishes the summary to an SNS topic. The code is as follows:

```
1 import json
2 import datetime
3
4 # Create AWS service clients
5 ssm = boto3.client('ssm')
6 sns = boto3.client('sns')
7
8 # Replace with your actual SNS topic ARN from the SNS console
9 SNS_TOPIC_ARN = 'arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts'
10
11 def lambda_handler(event, context):
12     # Extract alarm name from the EventBridge event
13     alarm_name = event['detail']['alarmName']
14
15     # Extract instance ID from the alarm's metric dimensions
16     instance_id = event['detail']['configuration']['metrics'][0]['metricStat']['metric']['dimensions']
17
18     # Send SSM Run Command to restart nginx on the affected instance
19     ssm.send_run_command(
20         InstanceIds=[instance_id],
21         DocumentName='AWS-RunShellScript',
22         Parameters={'commands': ['sudo systemctl restart nginx']}
23     )
24
25     # Build a remediation summary with timestamp
26     timestamp = datetime.datetime.now().isoformat()
27     message = {
28         f"Self-Healing Remediation Executed\n",
29         f"Alarm: {alarm_name}\n",
30         f"Instance: {instance_id}\n",
31         f"Action: Restarted nginx via SSM Run Command\n",
32         f"Timestamp: {timestamp}"
33     }
34
35     # Publish the summary to SNS
36     sns.publish(
37         TopicArn=SNS_TOPIC_ARN,
38         Subject=f'Self-Healing Alert: {alarm_name}',
39         Message=message
40     )
41
42     return {
43         'statusCode': 200,
44         'body': json.dumps('Remediation complete')
45     }
```

Lambda function code

How the Lambda function processes and responds to alarm events

When my function receives an event, it first extracts the alarm name and EC2 instance ID from the EventBridge alarm payload. In this step, I use that instance ID to send an SSM Run Command that remotely restarts the nginx service on the affected EC2 instance. Then the function creates a remediation summary with the alarm name, instance ID, action taken, and timestamp. Finally, it publishes this message to the SNS topic so I receive an email notification confirming the system automatically healed the failure.

Test event Info

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Edit saved event

Invocation type

☒ Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

AlarmStateChange

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

CloudWatch

Event JSON

```
1 {
2   "source": "aws.cloudwatch",
3   "detail-type": "CloudWatch Alarm State Change",
4   "detail": {
5     "alarmName": "HighCPU-SelfHealingDemo",
6     "state": {
7       "value": "ALARM",
8       "reason": "Threshold Crossed: 1 out of 1 datapoints were greater than the threshold (80.0) "
```

Lambda function test set up



☰ Lambda > Functions > SelfHealingRemediation ⓘ 🖨

🟢 The test event "AlarmStateChange" was successfully saved. ✕

Notifications 0 0 2 0 0 0 ▾

AlarmStateChange ▾ 🔄

Event JSON

[Format JSON](#)

```
1 {
2   "source": "aws.cloudwatch",
3   "detail-type": "CloudWatch Alarm State Change",
4   "detail": {
5     "alarmName": "HighCPU-SelfHealingDemo",
6     "state": {
7       "value": "ALARM",
8       "reason": "Threshold Crossed: 1 out of 1 datapoints were greater than the threshold (80.0).",
9       "timestamp": "2026-04-25T15:00:00.000+0000"
10    },
11    "previousState": {
12      "value": "OK",
13      "timestamp": "2026-04-25T14:55:00.000+0000"
14    },
15    "configuration": {
16      "metrics": [
17        {
18          "metricStat": {
19            "metric": {
20              "namespace": "AWS/EC2",
21              "name": "CPUUtilization",
22              "dimensions": {
23                "InstanceId": "i-0c11607843c0f048d"
24              }
25            }
26          }
27        }
28      ]
29    }
30  }
```

23:51 JSON

Lambda function test JSON

☰ Lambda > Functions > SelfHealingRemediation ⓘ 🖨

🟢 The test event "AlarmStateChange" was successfully saved. ✕

Notifications 0 0 2 0 0 0 ▾

🟢 Executing function: succeeded ([logs](#))

▼ Details

```
{
  "statusCode": 200,
  "body": "\\Remediation complete\\"
}
```

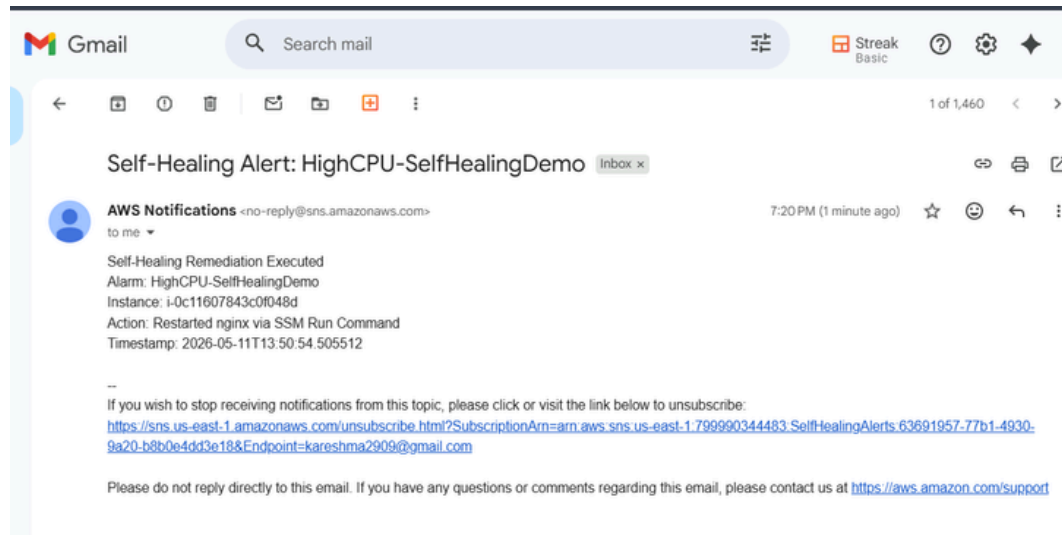
Summary

Code SHA-256 0mKUUFUEB2qgH1ZuzEBqOaxMWTlofQEBz7aGX9QH804Q=	Execution time 22 seconds ago
Function version \$LATEST	Request ID 43e83c1b-1d53-4c2e-ba19-f5cf6989e0c1
Duration 560.38 ms	Billed duration 1033 ms
Resources configured 128 MB	Max memory used 89 MB
Init duration 472.26 ms	

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: 43e83c1b-1d53-4c2e-ba19-f5cf6989e0c1 Version: $LATEST
END RequestId: 43e83c1b-1d53-4c2e-ba19-f5cf6989e0c1
REPORT RequestId: 43e83c1b-1d53-4c2e-ba19-f5cf6989e0c1  Duration: 560.38 ms    Billed Duration: 1033 ms    Memory Size: 128 MB    Max
Memory Used: 89 MB    Init Duration: 472.26 ms
```



SNS notification to email inbox

My Lambda function was created in ap-south-1 (Mumbai) while EC2, SNS, and CloudWatch were in us-east-1 (N. Virginia), causing an InvalidInstanceId error. All AWS resources in a pipeline must be in the same region. Always check the region dropdown before creating resources.

Wiring Up EventBridge to Route Events

Connecting the pipeline with event-driven routing

In this step, I'm creating an Amazon EventBridge rule that listens for my specific CloudWatch alarm event and routes it to my Lambda remediation function so that the system can automatically trigger the self-healing process whenever the alarm enters the ALARM state.



Amazon EventBridge

Rules

Create rule

Step 1

Define rule detail

Step 2

Build event pattern

Step 3

Select target(s)

Step 4 - optional

Configure tags

Step 5

Review and create

Review and create

Step 1: Define rule detail

Edit

Define rule detail

Rule name

HighCPU-Remediation-Rule

Description

Status

Enabled

Rule type

Standard rule

Event bus

default

Step 2: Build event pattern

Edit

Event pattern

Info

```
1 {
2   "source": ["aws.cloudwatch"],
3   "detail-type": ["CloudWatch Alarm State Change"],
4   "detail": {
5     "alarmName": ["HighCPU-SelfHealingWebserver"],
6     "state": {
7       "value": ["ALARM"]
8     }
9   }
10 }
```

Copy

EventBridge rule

Amazon EventBridge

Rules

Create rule

Copy

Step 3: Select target(s)

Edit

Targets

Details	Target Name	Type	ARN	Input	Role
▼	SelfHealingRemediation	Lambda function	arn:aws:lambda:us-east-1:799990344483:function:SelfHealingRemediation	Matched event	Amazon_EventBridge_Invoke_Lambda_374868651

Input to target:

Matched event

Additional parameters:

--

Dead-letter queue (DLQ):

-

Step 4: Configure tag(s)

Edit

Tags (0)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Value

No tags associated with this resource.

Cancel

Previous

Create rule

How the event pattern filters for the right triggers

The event pattern filters for events where the Amazon CloudWatch alarm state changes. It specifically checks that the event source is CloudWatch, the detail type is a CloudWatch Alarm State Change, and the alarm name matches HighCPU-SelfHealingWebserver. It also verifies that the alarm state value is ALARM. By defining these conditions, the rule ignores unrelated events and only captures the moment when this exact alarm enters the ALARM state. When this match happens, Amazon EventBridge triggers the AWS Lambda function to start the automated remediation process.

The screenshot shows the AWS EventBridge console. At the top, a green notification bar states: "Rule HighCPU-Remediation-Rule was created successfully". Below this, the "Rules" section is active, showing a list of "Event pattern rules (2)". A search bar contains the text "High", resulting in 1 match. The table below lists the rule:

	Name	Status	Type	Event bus	ARN	Description
☐	HighCPU-Remediation-Rule	Enabled	Standard	default	arn:aws:events:us-east-1:79999034483:rule/HighCPU-Remediation-Rule	-

EventBridge rule enabled

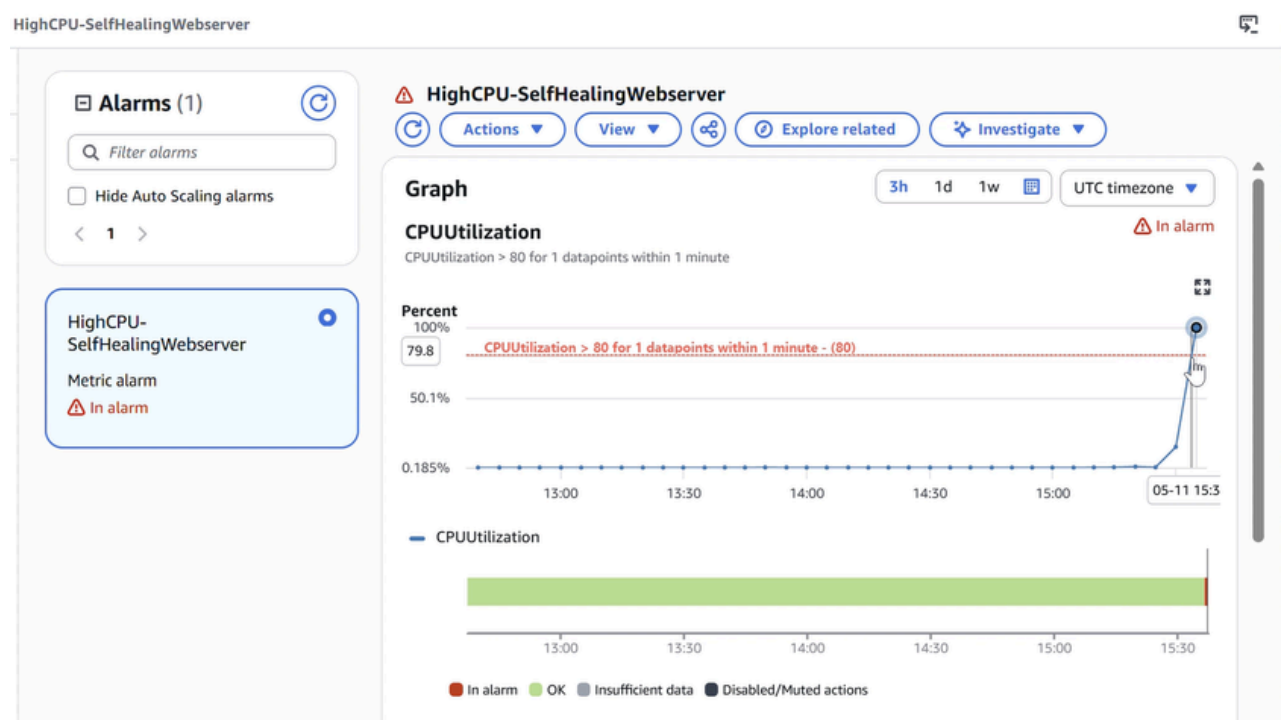
Proving the Self-Healing Pipeline Works End-to-End

Simulating failure and verifying automated recovery

In this step, I'm testing the entire self-healing pipeline by simulating a failure on my EC2 web server. I will intentionally stop nginx and create a CPU spike on the instance to trigger the Amazon CloudWatch alarm. This allows me to verify that the monitoring system detects the failure, Amazon EventBridge routes the alarm event, and the AWS Lambda function automatically restarts nginx and sends an alert through Amazon Simple Notification Service, confirming the self-healing system works end-to-end without manual intervention.

Testing the Self-Healing Pipeline

The pipeline was validated by intentionally stopping the nginx service and creating high CPU load using stress-ng. CloudWatch detected the anomaly, EventBridge triggered the Lambda function, and Systems Manager restarted nginx automatically. The system returned to a healthy state and a notification email confirmed the remediation.



CloudWatch In alarm after CPU spike

The full remediation chain from alarm to restored nginx

After the alarm fired, Amazon CloudWatch generated an alarm state change event when CPU utilization exceeded the configured threshold. Amazon EventBridge captured this event using the rule I created and routed it to my AWS Lambda remediation function. The Lambda function extracted the alarm and instance details from the event and used AWS Systems Manager to send a Run Command that restarted the nginx service on the EC2 instance. Once nginx was restarted successfully, the function published a message through Amazon Simple Notification Service, confirming the automated remediation and restoring the web server without manual intervention.

CloudWatch > Log management > /aws/lambda/SelfHealingRemediation > 2026/05/11/[\${LATEST}]c6db1ac495794f5184f9d22cd67fd5dc

CloudWatch <

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

1m 1h UTC timezone Display

Timestamp	Message
	No older events at this moment. Retry
2026-05-11T15:36:45.682Z	INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:e4...
2026-05-11T15:36:46.195Z	START RequestId: 612d94b4-dd76-4d93-9aac-4b7f230eadeb Version: \$LATEST
2026-05-11T15:36:46.742Z	END RequestId: 612d94b4-dd76-4d93-9aac-4b7f230eadeb
2026-05-11T15:36:46.742Z	REPORT RequestId: 612d94b4-dd76-4d93-9aac-4b7f230eadeb Duration: 546.45 ms Billed Duration: 1056 ms Memory Size...

No newer events at this moment. Auto retry paused. [Resume](#)

Log stream and events

Simulate a web server failure by stopping nginx and spiking CPU

stress-ng failed with "temp-path '.' must be readable and writeable" in Session Manager because the current directory lacked write access. Running "cd /tmp" before executing stress-ng resolved it, as /tmp is always readable and writeable on Linux instances.

```
Session ID: KareshmaR-fusp7tfce3kynkp4h9xcfsj5y Instance ID: i-Oc11607843cOf048d (SelfHealing-Webserver)
sh-5.2$ sudo systemctl stop nginx
sh-5.2$ sudo dnf install -y stress-ng
Last metadata expiration check: 0:09:18 ago on Mon May 11 15:22:48 2026.
Package stress-ng-0.15.05-1.amzn2023.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
sh-5.2$ stress-ng --cpu 2 --timeout 300
aborting: temp-path '.' must be readable and writeable
sh-5.2$ cd /tmp
sh-5.2$ stress-ng --cpu 2 --timeout 300
stress-ng: info: [8664] setting to a 300 second (5 mins, 0.00 secs) run per stressor
stress-ng: info: [8664] dispatching hogs: 2 cpu
```

☰

AWS Systems Manager > Run Command

Review node insights

Explore nodes

Diagnose and remediate

Just-in-time node access New

Settings

Node Tools

Compliance

Distributor

Fleet Manager

Hybrid Activations

Inventory

Patch Manager

Run Command

Session Manager

State Manager

Change Management Tools

Automation

Change Calendar

Try out the new AWS Systems Manager unified console

The unified console makes it easier to manage nodes across your organization — whether it's EC2 instances, hybrid servers, or servers running in a multicloud environment. [Learn more](#)

Get started

×

Commands

Command history

Command history

🔍 Search complete command history

View details

Rerun

Copy to new

Run command

Command ID	Status	Requested date	Document name	Comment	# of targets
fe43838f-556f-4328-9bfb-5af525030007	Success	Mon, 11 May 2026 15:36:46 GMT	AWS-RunShellScript		1
616d7dbb-489f-4163-b9db-d42833abbd19	Success	Mon, 11 May 2026 13:50:54 GMT	AWS-RunShellScript		1

System Manager – Command History



Gmail Search mail

ALARM: "HighCPU-SelfHealingWebserver" in US East (N. Virginia) Inbox x

AWS Notifications <no-reply@sns.amazonaws.com> to me
Mon, May 11, 9:06 PM (3 days ago)

You are receiving this email because your Amazon CloudWatch Alarm "HighCPU-SelfHealingWebserver" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [99.99584786580303 (11/05/26 15:35:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Monday 11 May, 2026 15:36:45 UTC".

View this alarm in the AWS Management Console:
<https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/HighCPU-SelfHealingWebserver>

Alarm Details:

- Name: HighCPU-SelfHealingWebserver
- Description:
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [99.99584786580303 (11/05/26 15:35:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Monday 11 May, 2026 15:36:45 UTC
- AWS Account: 799990344483
- Alarm Arn: arn:aws:cloudwatch:us-east-1:799990344483:alarm:HighCPU-SelfHealingWebserver

Threshold:

- The alarm is in the ALARM state when the metric is GreaterThanThreshold 80.0 for at least 1 of the last 1 period(s) of 60 seconds.

Monitored Metric:

- MetricNamespace: AWS/EC2
- MetricName: CPUUtilization
- Dimensions: [InstanceId = i-0c11607843c0f048d]
- Period: 60 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: missing

State Change Actions:

- OK:
- ALARM: [arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts]
- INSUFFICIENT_DATA:

SNS Email

Gmail Search mail

Self-Healing Alert: HighCPU-SelfHealingWebserver Inbox x

AWS Notifications <no-reply@sns.amazonaws.com> to me
9:06 PM (10 minutes ago)

Self-Healing Remediation Executed
Alarm: HighCPU-SelfHealingWebserver
Instance: i-0c11607843c0f048d
Action: Restarted os via SSM Run Command
Timestamp: 2026-05-11T15:36:46.521921

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
<https://sns.us-east-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts:63691957-77b1-4930-9a20-b8b0e4dd3e18&Endpoint=kareshma2909@gmail.com>

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

Extra: Auto-Restarting Stopped EC2 Instances

A second automation was implemented to restart EC2 instances that were stopped accidentally. A dedicated Lambda function checks whether an instance contains an AutoRestart tag before restarting it. This ensures only opted-in instances are automatically recovered.

IAM > Roles > Create role

Select trusted entity

Step 2

Add permissions

Step 3

Name, review, and create

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

Maximum 64 characters. Use alphanumeric and "+,=,@,-" characters.

Description
Add a short explanation for this role.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/\[\]!#\$%^&*~

Step 1: Select trusted entities

Trust policy

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "sts:AssumeRole"  
8       ],  
9       "Principal": {  
10        "Service": [  
11          "lambda.amazonaws.com"  
12        ]  
13      }  
14    ]  
15  }  
16 }
```

Step 2: Add permissions

Permissions policy summary

Policy name	Type	Attached as
AWSLambdaBasicExecutionRole	AWS managed	Permissions policy

IAM Permission to Auto Restart EC2 when stopped

Autorestart policy attach to AutorestartLambdaRole

IAM > Roles > AutorestartLambdaRole > Create policy

Step 1
Specify permissions

Step 2
Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name
Enter a meaningful name to identify this policy.

AutoRestartPolicy

Maximum 128 characters. Use alphanumeric and "+", "@", "-" characters.

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it.

Search

Allow (2 of 469 services) ☐ Show remaining 467 services

Service	Access level	Resource	Request condition
EC2	Limited: List, Write	All resources	None
SNS	Limited: Write	TopicName string like [SelfHealingAlerts, region] string like [All]	None

Cancel Previous Create policy

IAM Permission policy to Auto Restart EC2 when stopped

IAM > Roles > AutorestartLambdaRole > Create policy

Step 1
Specify permissions

Step 2
Review and create

Specify permissions

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

Visual | JSON | Actions

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "ec2:StartInstances",
8         "ec2:DescribeTags"
9       ],
10      "Resource": "*"
11    },
12    {
13      "Effect": "Allow",
14      "Action": "sns:Publish",
15      "Resource": "arn:aws:sns:*:*:SelfHealingAlerts"
16    }
17  ]
18 }
```

Edit statement

Select a statement

Select an existing statement or add a new statement

+ Add new statement

AutoRestart Lambda function and code

[Lambda](#) > [Functions](#) > Create function

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

AutoRestartInstance

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.12

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **More settings**.

[Lambda](#) > [Functions](#) > Create function

▼ **Additional settings**
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

General [Info](#)

ARM64 architecture ☐
Use ARM64 instead of x86_64 (default).

Custom execution role [Edit](#) ☒
Use custom execution role instead of new role with basic Lambda permissions (default).

IAM permissions
[AutoRestartLambdaRole](#)

[Lambda](#) > [Functions](#) > [AutoRestartInstance](#) > Edit environment variables

Edit environment variables

Environment variables
You can define environment variables as key-value pairs that are accessible from your function code. These are useful to store configuration settings without the need to change function code. [Learn more](#)

Key	Value	
SNS_TOPIC_ARN	arn:aws:sns:us-east-1:799990344483:SelfHealing	Remove

[Add environment variable](#)

► **Encryption configuration**

[Cancel](#) [Save](#)

☰ Lambda > Functions > AutoRestartInstance ⓘ 🖨

✔ Successfully updated the function "AutoRestartInstance". ✕

AutoRestartInstance

Throttle Copy ARN Actions

▼ Function overview ⓘ

Diagram | Template

AutoRestartInstance
Layers (0)

+ Add trigger + Add destination

Export to Infrastructure Composer Download

Function ARN
arn:aws:lambda:us-east-1:799990344483:function:AutoRestartInstance

Description
-

Last modified
23 seconds ago

Code | Test | Monitor | **Configuration** | Aliases | Versions

Configuration

General configuration ⓘ Edit

Description
-

Timeout
0 min 30 sec

Memory
128 MB

SnapStart ⓘ
None

Ephemeral storage
512 MB

Lambda function configuration

EXPLORER

- ▼ AUTORESTARTINSTANCE
 - lambda_function.py
- ▼ DEPLOY ▾ Undeployed
 - Deploy (Ctrl+Shift+U)
 - Test (Ctrl+Shift+I)
- ▼ TEST EVENTS [NONE SELECTED]
 - + Create new test event

lambda_function.py X

```
1 import boto3
2 import os
3 from datetime import datetime
4
5 def lambda_handler(event, context):
6     # Extract instance ID and state from the EC2 state change event
7     instance_id = event['detail']['instance-id']
8     state = event['detail']['state']
9
10    print(f"Instance {instance_id} changed to state: {state}")
11
12    # Check if the instance has the AutoRestart tag set to "true"
13    ec2 = boto3.client('ec2')
14    tags_response = ec2.describe_tags(
15        Filters=[
16            {'Name': 'resource-id', 'Values': [instance_id]},
17            {'Name': 'key', 'Values': ['AutoRestart']}
18        ]
19    )
20
21    # Only proceed if the tag exists and its value is "true"
22    auto_restart = False
23    for tag in tags_response['Tags']:
24        if tag['Key'] == 'AutoRestart' and tag['Value'] == 'true':
25            auto_restart = True
26
27    if not auto_restart:
28        print(f"Instance {instance_id} does not have AutoRestart=true. Skipping.")
29        return {'statusCode': 200, 'body': 'Skipped - no AutoRestart tag'}
30
31    # Restart the stopped instance
32    ec2.start_instances(InstanceIds=[instance_id])
33    print(f"Restart initiated for instance {instance_id}")
34
35    # Send a notification to the SelfHealingAlerts SNS topic
36    sns = boto3.client('sns')
37    topic_arn = os.environ['SNS_TOPIC_ARN']
38    timestamp = datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S UTC')
39
40    message = (
41        f"Auto-Restart Triggered\n"
42        f"Instance ID: {instance_id}\n"
43        f"Previous State: {state}\n"
44        f"Action: Instance restart initiated\n"
45        f"Reason: Instance was stopped and has AutoRestart=true tag\n"
46        f"Timestamp: {timestamp}"
47    )
48
49    sns.publish(
50        TopicArn=topic_arn,
51        Subject=f'Auto-Restart: {instance_id}',
52        Message=message
53    )
54
55    return {'statusCode': 200, 'body': f'Restarted instance {instance_id}'}
```



Amazon EventBridge > Rules > Create rule

Builder mode

☐ Enhanced builder
Visual drag-and-drop builder

☒ Advanced builder
Step-by-step rule configuration

Feedback

Step 1
Define rule detail

Step 2
Build event pattern

Step 3
Select target(s)

Step 4 - optional
Configure tags

Step 5
Review and create

Review and create

Step 1: Define rule detail

Edit

Define rule detail

Rule name EC2-AutoRestart-Rule	Status Enabled	Event bus default
Description	Rule type Standard rule	

Step 2: Build event pattern

Edit

Event pattern info

```
1 {
2   "source": ["aws.ec2"],
3   "detail-type": ["EC2 Instance State-change Notification"],
4   "detail": {
5     "state": ["stopped"]
6   }
7 }
```

Copy

EventBridge rule stopped event

Amazon EventBridge > Rules > Create rule

```
4   "detail": {
5     "state": ["stopped"]
6   }
7 }
```

Copy

Step 3: Select target(s)

Edit

Targets

Details	Target Name	Type	ARN	Input	Role
▶	AutoRestartInstance	Lambda function	arn:aws:lambda:us-east-1:79990344483:function:AutoRestartInstance	Matched event	Amazon_EventBridge_Invoke_Lambda_1360185502

Step 4: Configure tag(s)

Edit

Tags (0)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value
No tags associated with this resource.	

Cancel Previous Create rule

Request to manage tags has succeeded.

Instances (1/1) Info

Last updated about 1 hour ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

⚙️

☒	Name	Instance ID	Instance state	Instance type	Status check	Alar
☒	SelfHealing-Webserver	i-0c11607843c0f048d	Running	t3.micro	3/3 checks passec	View

i-0c11607843c0f048d (SelfHealing-Webserver)

⚙️

⌵

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

Tags

Manage tags

Q

< 1 >

⚙️

Key	Value
AutoRestart	true

Name SelfHealing-Webserver

Tag Instance

Tag-based filtering for safe production automation

In this project extension, I added a tag check that uses Amazon EC2 instance tags to determine whether an instance should be restarted automatically. The AWS Lambda function receives the stop event, then calls the EC2 API to read the instance tags and look for AutoRestart=true. If the tag exists, the function starts the instance; if not, it safely skips the action.

This filter is important in production because not every stopped instance should be restarted. Some are intentionally stopped for maintenance, cost control, or decommissioning. The tag creates an explicit opt-in mechanism so only selected instances are automatically restarted.



Successfully initiated stopping of i-Oc11607843c0f048d

Notifications 0 0 2 0 0 0

Instances (1/1) Info

Last updated about 1 hour ago

Connect Instance state Actions Launch instances

Saved filter sets Choose filter set

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
SelfHealing-Webserver	i-Oc11607843c0f048d	Stopping	t3.micro	3/3 checks passed	View alarms	us-east-1

i-Oc11607843c0f048d (SelfHealing-Webserver)

Details Status and alarms Monitoring Security Networking Storage Tags

▼ Instance summary Info

Instance ID i-Oc11607843c0f048d	Public IPv4 address 34.205.55.131 open address	Private IPv4 addresses 172.31.7.123
IPv6 address -	Instance state Stopping	Public DNS ec2-34-205-55-131.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-7-123.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-7-123.ec2.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations.
Auto-assigned IP address 34.205.55.131 [Public IP]	VPC ID vpc-04abc8deffc8f5072	

Instance stopped to fire event rule

Successfully initiated stopping of i-Oc11607843c0f048d

Notifications 0 0 2 0 0 0

Instances (1/1) Info

Last updated less than a minute ago

Connect Instance state Actions Launch instances

Saved filter sets Choose filter set

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
SelfHealing-Webserver	i-Oc11607843c0f048d	Running	t3.micro	Initializing	View alarms	us-east-1

i-Oc11607843c0f048d (SelfHealing-Webserver)

Details Status and alarms Monitoring Security Networking Storage Tags

▼ Instance summary Info

Instance ID i-Oc11607843c0f048d	Public IPv4 address 13.220.43.141 open address	Private IPv4 addresses 172.31.7.123
------------------------------------	---	--



Log management > /aws/lambda/AutoRestartInstance > 2026/05/11/[LATEST]969da2a62047481ab2d67b629fdb245a

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

1m 1h UTC timezone Display

Timestamp	Message
2026-05-11T16:35:03.148Z	INIT_START Runtime Version: python:3.12.mainlinev2.v7 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:4ab553846c4e081013ff7d1d608a5358d5b956bb5b1c83c66d2a31da8f6244
2026-05-11T16:35:03.505Z	START RequestId: 5e4e5eb7-46b6-4401-bd2b-d44ba141b0c8 Version: \$LATEST
2026-05-11T16:35:03.506Z	Instance i-0c11607843c0f048d changed to state: stopped
2026-05-11T16:35:08.258Z	Restart initiated for instance i-0c11607843c0f048d
2026-05-11T16:35:08.633Z	END RequestId: 5e4e5eb7-46b6-4401-bd2b-d44ba141b0c8
2026-05-11T16:35:08.633Z	REPORT RequestId: 5e4e5eb7-46b6-4401-bd2b-d44ba141b0c8 Duration: 5127.43 ms Billed Duration: 5481 ms Memory Size: 128 MB Max Memory Used: 97 MB Init Duration: 352.83 ms

Back to top

Log groups

Gmail Search mail

Auto-Restart: i-0c11607843c0f048d

AWS Notifications <no-reply@sns.amazonaws.com> May 11, 2026, 10:05 PM (19 hours ago)

to me

Auto-Restart Triggered
Instance ID: i-0c11607843c0f048d
Previous State: stopped
Action: Instance restart initiated
Reason: Instance was stopped and has AutoRestart=true tag
Timestamp: 2026-05-11 16:35:08 UTC

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
<https://sns.us-east-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:799990344483:SelfHealingAlerts:63691957-77b1-4930-9a20-b8b0e4dd3e18&Endpoint=kareshma2909@gmail.com>

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

Email notification for Autorestart Instance



Key Learnings

- Designing event driven automation pipelines
- Using CloudWatch alarms for operational monitoring
- Automating remediation using AWS Lambda
- Using Systems Manager for remote command execution
- Implementing guardrails with tag based automation

Future Improvements

- Extend the system with AI-driven incident analysis using Amazon Bedrock
- Observability dashboards
- DevOps Copilot interface for querying infrastructure health
- Additional health checks for nginx
- Auto Scaling integration
- Implement automated rollback strategies
- Add centralized logging dashboards
- Integrate Slack or incident management tools for team alerts



Cost Considerations

- The architecture is designed around AWS Free Tier services.
- EC2 t3.micro: Free tier eligible
- Lambda: Always free tier usage
- CloudWatch alarms: Free tier limits
- EventBridge: Free tier event volume
- SNS notifications: Minimal cost

Cleanup

To avoid charges after experimentation:

1. Terminate EC2 instance
2. Delete Lambda functions
3. Remove EventBridge rules
4. Delete SNS topic and subscription
5. Delete CloudWatch alarm
6. Remove IAM roles
7. Delete security group allowing HTTP on port 80.



Kareshma
Rajaananthapadmanaban

*Follow up for more
interesting projects !!!*



Check out my profile for more projects